

Summarized note based on <https://youtu.be/9YTNYT1maa4?si=IUX5YchAL5uBhFh7>

- Data visualization and data science, keynote lecture by Hadley Wickham

The idea behind the Grammar of Graphics is that any visualization can be broken down into small pieces or components, and by combining these pieces in different ways, we can create different types of visualizations. Hadley Wickham created open-source R packages, including ggplot package and later ggplot2, to make Grammar of Graphics-based visualization easier. As he created more and more visualizations, he found that the main difficulty was often not creating the visualization but getting the data into the right format. This led to his work on data reshaping and data manipulation, including packages such as tidyr, dplyr, and purrr, which are part of the tidyverse (with plyr being an earlier related package). The idea of tidy data provides a standard way to organize data so that the meaning of the data is consistent: each column represents a variable and each row represents an observation. For example, the Gapminder dataset used in Hans Rosling's visualizations contains variables such as country, region/continent, year, population, income, and life expectancy. Therefore, before creating a visualization, we should first organize the data into a tidy format, because well-structured data makes it easier to analyze and visualize information using R and ggplot.

He first organizes the Gapminder data by selecting the year he wants to examine so that he can work with a specific, manageable subset of the data. He then creates a basic scatter plot to show the relationship between income and life expectancy. Next, he changes the income scale to a logarithmic scale because income values vary greatly between countries, making the pattern easier to see and comparable to the original visualization. He then adds color to represent regions and point size to represent population, allowing more variables to be shown in the same graph. After creating the basic visualization, he polishes it by adjusting the axis ranges, marks, labels, and colors to make the graph clearer and easier to interpret. He also fixes overplotting because large countries can cover smaller countries; therefore, he plots the larger countries first so that smaller countries remain visible. Rather than creating a separate graph for every year, he then selects multiple years to examine changes over time. He creates individual plots for these years and then demonstrates faceting to display several years together for comparison, although the plots become smaller. Finally, he uses animation to show how the visualization changes from year to year, making the temporal pattern easier to understand. He also mentions packages such as ggseq, which can be combined with animation for specialized visualizations (example: brain map). Finally, he demonstrates rendering the document directly into PDF, HTML, or DOC/DOCX so that the code, visualizations, and explanation can be compiled together without manually copying and pasting.

Building Visualizations with ggplot2

Organize

First, organize the data by taking the gapminder dataset and filtering it for the year 2015:

```
gapminder15 <- gapminder %>%  
  filter(year == 2015)
```

- gapminder → take the Gapminder data.
- %>% → then pass the data to the next step.
- filter(year == 2015) → keep only the observations from 2015.
- gapminder15 → create a new variable containing the filtered data.

Visualization Steps

Step 1. Create the scatter plot.

Create a scatter plot showing the relationship between income and life expectancy.

Step 2. Match the scale of the original plot.

Use a logarithmic scale for income to match the scale of the original visualization.

Step 3. Add colors and sizes.

Add colors to represent the four regions and use population size to control the size of the points.

```
gapminder %>%  
  ggplot(aes(income, life_exp)) +  
  geom_point(aes(color = region, size = pop)) +  
  scale_x_log10()
```

Then polish the visualization by changing the x- and y-axis scale ranges and marks, adding x- and y-axis labels, and changing the colors to fixed colors.

Fixing Overplotting

There may be smaller countries whose points cannot be seen because they are covered by larger points representing countries with larger populations. To fix this, plot the countries in an appropriate order so that the **larger countries are in the back** and the smaller countries can remain visible.

Showing All Years

Rather than filtering by one year every time, we can work with multiple years and show the changes over time.

First, select every fifth year and split the data by year:

```
by_year <- gapminder %>%  
  filter(year %% 5 == 0) %>%  
  group_split(year)
```

Then create a plot for each year:

```
plots <- map(by_year, ~ gap_plot(.x))
```

This gives us a bunch of plots, one for each selected year.

Faceting -Another approach is faceting, which displays multiple plots for different years at the same time.

```
gapminder5 %>%  
  gap_plot("All years") +  
  facet_wrap(~ year) +  
  scale_size(range = c(0, 4), guide = FALSE)
```

This gives us a bunch of plots, but they may not be very clear because the individual plots become smaller.

Animating

Instead of creating many separate plots, we can animate the visualization over time using the **gganimate** package.

```
library(gganimate)  
animation <- gapminder5 %>%  
  gap_plot("Year: {frame_time}") +  
  transition_time(year)
```

- `transition_time(year)` → changes the visualization according to the year.
- `{frame_time}` → displays the current year in the animation.

There are also other packages, such as **ggseq**, which can be used to work with brain atlases and can be combined with **gganimate** to show changes through animation.

Rendering the Document

Instead of copy-pasting the code and output manually, we can **render the document directly** into formats such as:

- **PDF**
- **HTML**
- **DOC/DOCX**